

LangChain and Retrieval-Augmented Generation

Technical reference document for Retrieval-Augmented Generation (RAG) practice.

1. LangChain Overview

LangChain is a framework for building applications around language models. It provides abstractions for prompts, model calls, document processing, retrieval, tools, and application workflows.

- Document loaders ingest information from files or other sources.
- Text splitters divide large documents into smaller chunks.
- Retrievers select relevant documents for a query.
- Chains and runnable components connect processing steps.

2. Documents and Document Loaders

A document generally contains page or text content plus metadata. A loader converts an external source into document objects that can be processed by a pipeline.

- PDF loaders can extract text from PDF files.
- Metadata can include a source path, page number, title, or document identifier.
- Keeping source metadata enables citations and debugging during retrieval.

3. Text Splitting

Language-model context windows are limited, and retrieval usually works better with focused passages. Text splitters divide documents into chunks while optionally using overlap to preserve context across boundaries.

- Chunk size controls the approximate amount of text in each chunk.
- Chunk overlap repeats a small amount of neighboring content.
- Recursive splitting attempts to preserve natural boundaries such as paragraphs and sentences.

4. Embeddings and Vector Stores

An embedding model converts text into a numerical vector. A vector store indexes those vectors so that a query can be compared with stored document vectors.

- Cosine similarity is commonly used to compare embedding directions.
- Metadata filters can restrict retrieval to selected sources or categories.
- The embedding model used for documents and queries must be compatible.

5. Retrievers

A retriever accepts a query and returns relevant documents. In a basic vector retriever, similarity search finds chunks whose embeddings are close to the query embedding.

- Top-k retrieval returns a selected number of candidate chunks.
- Retrieval quality can be improved with metadata filtering, reranking, or hybrid lexical and semantic search.
- Too many retrieved chunks can add noise to the generation prompt.

6. Prompt Templates

Prompt templates define reusable instructions and variables. In a RAG workflow, retrieved context is inserted into a prompt along with the user's question.

- System instructions can define behavior and constraints.
- Context should be clearly separated from the question.
- Prompts can instruct the model to answer only from supplied context when grounded answers are required.

7. RAG Chain

A basic RAG pipeline has two major stages: retrieval and generation. The user query is converted into a retrieval query, relevant chunks are selected, and the chunks are supplied to a language model to produce an answer.

- Ingestion: load, split, embed, and store documents.
- Retrieval: embed the query and retrieve relevant chunks.
- Generation: combine the question and retrieved context in a prompt.
- Evaluation: measure retrieval relevance and answer correctness.

8. Example RAG Workflow

A practical workflow can be represented as: PDF files → document loader → text splitter → embedding model → vector database → retriever → prompt template → chat model → answer.

- Keep ingestion code separate from query-time code.
- Persist the vector index so documents do not need to be embedded on every application start.
- Log retrieved sources while developing so retrieval failures are visible.